

Executive Summary

Modern data applications increasingly require analyzing **billions to trillions of rows** of data with high ingest rates and long retention. Traditional single-node databases cannot scale to this volume, so *distributed database systems* (data warehouses, lakehouses, and NoSQL clusters) are used. These systems combine massively parallel processing (MPP) architectures, columnar storage, and advanced query optimizations to handle large-scale workloads. Key techniques include *intelligent data partitioning and sharding*, specialized *distributed join algorithms*, and *indexing/data layout optimizations* (e.g. columnar storage, skip indexes, bloom filters) to minimize I/O. Additional strategies like *predicate pushdown*, *vectorized execution*, and *cost-based query planning* further improve performance. Materialized aggregates (pre-aggregations, materialized views, approximate algorithms) reduce query work, while effective query routing and partition pruning ensure only relevant data is scanned. Real-world examples (e.g. an Iceberg table with 100B+ rows on AWS) show that careful partition design and storage formats can cut costs ~75% ¹ ². This white paper surveys these topics in depth, with SQL/DDDL examples, diagrams, and citations from vendor documentation and academic sources. We conclude with best-practice recommendations for building and tuning distributed databases at billion+ row scale.

Problem Statement and Scale

Many organizations collect massive data volumes (e.g. time-series logs, user events, IoT sensor data, genomics) reaching **billions to trillions of rows**. For example, Ancestry manages a “Hints” dataset of 100+ billion rows with 7 million hourly changes ¹. These systems must support high **ingestion rates** (millions of new rows per hour) and store data for long **retention** periods (often years). Queries range from simple lookups to complex analytic joins and aggregations. Key challenges include:

- **Volume:** Data size often spans terabytes to petabytes. OLAP workloads on such data need high I/O throughput and parallelism ³.
- **Velocity:** Data may arrive in real time (streaming) or in large batch loads (e.g. daily ETL). Systems may use Lambda or Kappa architectures to handle batch vs streaming layers ⁴ ⁵.
- **Variety:** Data may be relational, semi-structured (JSON), or key-value. Many systems must coexist (data lake + data warehouse).
- **Consistency vs Performance:** OLTP systems require ACID for real-time transactions, while OLAP systems prioritize read throughput and can tolerate eventual consistency on large aggregated data ⁶.

At these scales, single-server databases hit limits. Distributed architectures partition data across nodes and use MPP query engines. But performance depends heavily on data layout and query planning. Without careful design, even simple queries can require full-table scans or massive shuffles. Thus, **query optimization** for distributed systems must leverage partitioning, indexing, and execution strategies that minimize data movement and I/O.

Architecture Patterns

Distributed data systems follow different architectural patterns:

- **OLTP vs OLAP.** OLTP databases handle high-volume transactions (millions of small writes, e.g. updates, inserts) and prioritize write throughput and ACID consistency. OLAP systems handle analytic queries (large scans, aggregations) on historical data, prioritizing read throughput. OLAP stores often use columnar layouts and can accept eventual consistency, whereas OLTP uses row storage and requires strict consistency ⁷ ⁸ . Storage needs differ: OLTP might be GB-TB scale, OLAP tends to be TB-PB scale ³ . In practice, a hybrid architecture often combines an OLTP source (e.g. Postgres, Cassandra) feeding an OLAP warehouse (e.g. BigQuery, Snowflake) for analysis.
- **Lambda vs Kappa.** Many large-data systems ingest both batch and streaming data. *Lambda architecture* separates batch (historical processing) and speed (real-time processing) layers. For example, raw data lands on HDFS (or S3) for batch analytics, while a stream pipeline (e.g. Kafka + Spark Streaming) provides low-latency updates ⁴ ⁵ . The two layers merge in a serving layer to answer queries. *Kappa architecture* simplifies this by treating all data as a stream: a single pipeline (often Kafka + a stream processor like Flink/Spark) handles both live and replayed historical data ⁴ ⁹ . Kappa can be simpler but may incur higher storage and complexity in long-term streaming. Choice depends on use case: Lambda is chosen when both high accuracy (batch recomputation) and real-time analytics are needed (e.g. fraud detection), whereas Kappa suits pure real-time analytics with evolving business logic.
- **Data Lake + Query Engines.** Many organizations use a *data lake* (cloud object storage like S3, ADLS) to store raw data in open formats (Parquet, ORC, Iceberg, Delta). A metadata catalog (like AWS Glue, Hive Metastore, Iceberg catalog) tracks table schemas. Query engines (e.g. Presto/Trino, Spark, Snowflake, BigQuery, ClickHouse) run SQL queries over this data. For instance, Ancestry built an S3-based Iceberg table for 100B rows and uses EMR (Spark) to process it ¹⁰ . Other patterns include cloud warehouses (BigQuery or Snowflake) that manage storage and compute for you. Hybrid architectures (data mesh or lakehouse) often mix object storage with engines that can read it (e.g. Athena on S3, Databricks Delta Lake).
- **Serverless vs Provisioned.** Some engines are serverless (BigQuery, Athena) and automatically scale, charging per byte scanned. Others (Redshift, Snowflake, ClickHouse) use provisioned clusters. The choice affects how you optimize: e.g. BigQuery cost depends on data scanned (so partitioning/ clustering reduces cost), whereas provisioned clusters require provisioning enough nodes for peak load.

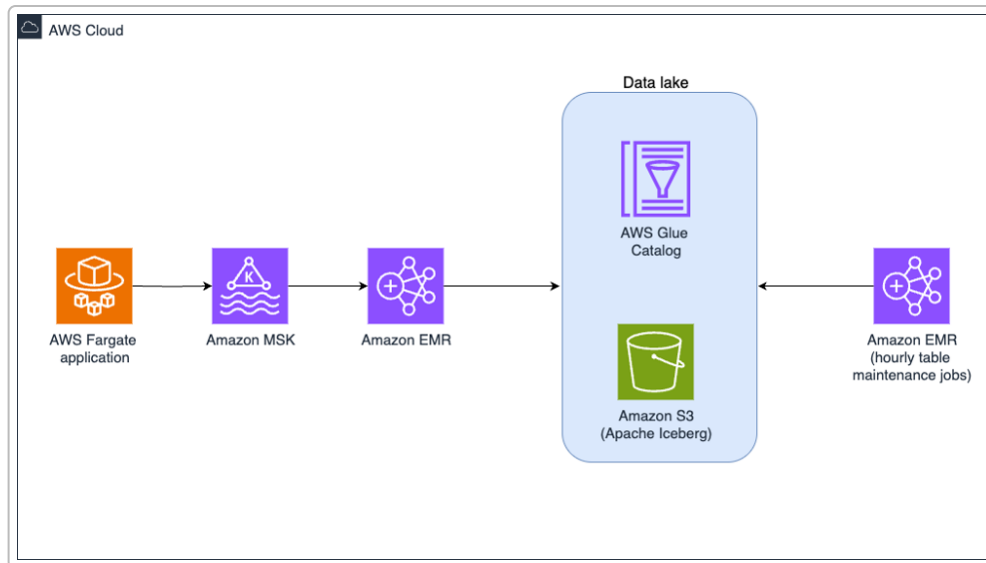


Figure: Example distributed data architecture (Ancestry's data lake pipeline on AWS) ¹⁰. In this design, Kafka (MSK) ingests events, Amazon EMR processes and writes to an Iceberg table on S3, and AWS Glue Catalog tracks metadata. This “single source of truth” lake houses the 100B+ row dataset for downstream analytics ¹⁰.

Partitioning and Sharding

Partitioning schemes divide data within a table into distinct segments. Common schemes:

- **Range partitioning:** each partition covers a continuous range of values (e.g. date ranges). Good for time-series or sorted data. Example (PostgreSQL):

```
CREATE TABLE sales (...) PARTITION BY RANGE (order_date);
CREATE TABLE sales_2023q1 PARTITION OF sales
FOR VALUES FROM ('2023-01-01') TO ('2023-04-01');
```

Range splits allow quick elimination of partitions outside the query's range. Vendors like BigQuery use time-based partitioning similarly ¹¹. However, uneven data distribution (e.g. if recent months get much more data) can cause skew.

- **List partitioning:** each partition holds specific discrete values (e.g. region or category). In PostgreSQL:

```
CREATE TABLE orders (...) PARTITION BY LIST (region);
CREATE TABLE orders_east PARTITION OF orders FOR VALUES IN ('US-EAST', 'EU-CENTRAL');
```

List is useful for categorical sharding (regions, product lines). It allows targeted pruning on those values, but requires enumerating partitions up front.

- **Hash partitioning:** data is distributed by hashing a key column modulo N. Guarantees even distribution across partitions, preventing hotspots ¹² ¹³. Example (PG 11+):

```
CREATE TABLE students (...) PARTITION BY HASH (student_id);  
CREATE TABLE students_p0 PARTITION OF students FOR VALUES WITH (MODULUS 5,  
REMAINDER 0);
```

...

Hash shards are excellent for balancing load, but range queries on the hash key are not efficient. ClickHouse and many NoSQL systems effectively use hash-based sharding internally.

- **Composite partitioning** (multi-level): combine methods, e.g. first range by date, then list by region within each. Snowflake's Iceberg example used "two-level partitioning": first by `status`, then by `type` ². Composite/iceberg transforms (YEAR(date), TRUNC(num, n), BUCKET(id)...) can tune partitions.
- **Time-based partitions:** a subtype of range/list: often one partition per day/month/year on a timestamp column. BigQuery and Snowflake use date partitions to isolate data by date ¹¹. ClickHouse often partitions MergeTree tables by month (via `PARTITION BY toYYYYMM(date_col)`) ¹⁴. Time partitions allow dropping old data easily and efficient date filtering.
- **Multi-dimensional:** Partition on multi-dimensional keys (e.g. spatial or range grid). Some systems support space-filling curves (e.g. Z-order) or user-managed shards on multiple columns. These are advanced and often achieved with manual hashing of composite keys.
- **Consistent hashing:** In systems like Cassandra, sharding is implicit via a hash ring on the partition key. Cassandra's partitioner hashes the partition key to tokens and assigns ranges to nodes ¹⁵. This yields horizontal scaling without manually defined partitions. Consistent hashing also facilitates adding/removing nodes with minimal reshuffling.

Sharding refers to distributing partitions across nodes. Some systems (like Citus/Postgres, MySQL Cluster, Amazon Redshift) let you pick a distribution key (often a hash), placing rows with the same key on the same shard. Others (ClickHouse's "Distributed" engine, BigQuery) handle it transparently. Cassandra treats the partition key (or first column of a compound primary key) as the shard key, with a built-in ring. For example, in Cassandra:

```
CREATE TABLE logs (user_id UUID, ts TIMESTAMP, event TEXT, PRIMARY KEY  
((user_id), ts));
```

Each unique `user_id` (the partition key) is hashed and placed on a node ¹⁵. Avoiding "hot" partitions is crucial: Cassandra recommends keeping each partition <100MB ¹⁶ by, e.g. including a time bucket in the key.

Vendor-specific examples:

- **PostgreSQL** supports RANGE, LIST, HASH partitioning (and “RANGE BY LIST(expression)” for limited multi-level cases) ¹⁷ [\[12†L443-L452\]](#) ¹². Example: quarterly range partitions on `joined_date` ¹⁷, or hash-sharding a table into N partitions ¹². PG also supports subpartitioning (e.g. each range partition can be further list-partitioned).
- **ClickHouse** uses its *MergeTree* engines. It typically partitions by month or year on a date column (e.g. `PARTITION BY toYYYYMM(event_date)`) ¹⁸. (Note: ClickHouse docs caution that partitions “do not speed up queries” beyond coarse elimination, so often monthly is enough ¹⁴.) The *ORDER BY* clause in MergeTree is effectively a primary key: ClickHouse stores data sorted by that key to enable zone-map pruning within each partition ¹⁹ ²⁰. For example:

```
CREATE TABLE events (  
  event_date Date,  
  user_id UInt64,  
  action String  
) ENGINE = MergeTree()  
  PARTITION BY toYYYYMM(event_date)  
  ORDER BY (user_id, event_date);
```

Here, partitions are monthly chunks, and within each, data is sorted by `user_id, event_date` for fast lookups. ClickHouse also supports *Distributed* tables to unify multiple shards, but the storage engine (MergeTree) handles partitions as above ²⁰.

- **BigQuery** allows time-unit column partitioning (daily/monthly on a DATE/TIMESTAMP column) and integer range partitioning ¹¹. For example:

```
CREATE TABLE mydataset.sales  
  PARTITION BY DATE(order_ts);
```

This creates daily partitions. BigQuery also has ingestion-time partitions (`_PARTITIONTIME`) automatically if chosen ²¹. BigQuery **clustering** can further sort data within partitions on up to 4 columns, helping prune blocks ²².

- **Snowflake** automatically micro-partitions data (50–500MB contiguous columnar chunks) ²³ ²⁴. These act as partitions: each has metadata (min/max for each column). Snowflake also allows defining clustering keys on columns to co-locate similar values in contiguous micro-partitions (improving pruning) ²⁵. Snowflake automatically “partitions” data; you typically do not manually split tables.
- **Cassandra** uses a *partition key* (possibly composite) defined in the PRIMARY KEY. The first column(s) is hashed by the partitioner. All rows with the same partition key go on one node (a “wide row” sorted by clustering columns). E.g. `PRIMARY KEY ((customer_id), order_date)` puts each customer’s orders on the same shard. Cassandra also uses consistent hashing across nodes ¹⁵.

Each scheme has trade-offs: range partitions are good for range queries, hash for even load, list for categorical queries. Too many partitions (e.g. daily over decades) can slow planning, and too few can force

large scans. Effective partitioning usually balances query patterns (filtering columns) against manageability

26 27 .

Distributed Join Strategies

Joining large tables across nodes is expensive, so distributed systems use specialized strategies:

- **Broadcast (Replicated) Join:** If one table is much smaller (fits in memory), it is *broadcasted* to all nodes, so each node can join locally with partitions of the large table 28 . This avoids shuffling the big table. For example, in Spark: `largeDF.join(broadcast(smallDF), "key")` will send `smallDF` to all executors 28 . This is ideal when **build side << build-side threshold** (Spark default ~10MB 29). Many engines (Presto, Spark, Snowflake) do this automatically.
- **Shuffle (Partitioned) Join:** If both tables are large, the engine typically repartitions (hash-distributes) both tables on the join key and does a parallel local join on each shard. This involves a *shuffle* (repartition) step. In Spark, this is a “Shuffle Hash Join” or “Sort-Merge Join” depending on planner. It’s costly but necessary when no small build side. Ensuring the join key is well-chosen (common cardinality and distribution) can help avoid skew.
- **Colocated (Local) Join:** When two tables share the same partitioning/sharding key and are stored on the same cluster nodes, the join can be done without a shuffle. Some systems support co-partitioned tables. For example, in a distributed Postgres (Citus), if both tables are distributed by `customer_id`, then joining on that key is local to each shard. In Hive/Impala, *bucketed* tables (same number of buckets and same bucketing key) enable bucketed joins. Similarly, in Iceberg with SPJ (storage-partitioned join), if both tables are Iceberg-partitioned identically on the join key, Iceberg can eliminate the shuffle 30 .
- **Adaptive Joins:** Modern engines like Spark (with AQE) can switch strategies mid-query. If one side’s size is mis-estimated, Spark can dynamically convert a shuffle join to a broadcast join if the runtime sizes are small enough 31 .
- **Example Usage:** Broadcast joins shine when joining a multi-billion row fact table with a few-thousand-row dimension table. Shuffle joins are needed for two fact tables. Colocated joins require upfront planning: ensuring the same shard key, or using a shuffle partitioner to align partitions.

Indexing and Data Layout

The way data is stored and indexed dramatically affects query performance:

- **Row vs Column Stores:** Traditional RDBMS (Postgres, MySQL) use *row-oriented* storage: each row is stored together. This is efficient for OLTP (reading/writing individual rows) but scans of many rows (common in analytics) waste I/O (reading unused columns). Columnar stores (Snowflake, BigQuery, ClickHouse, Redshift) store each column separately. For analytic queries that select a few columns over many rows, columnar reduces I/O and allows high compression (e.g. run-length, dictionary). For instance, ClickHouse’s `Wide` format stores each column in its own file for fast column reads 32 .
- **Compression and Encoding:** Columnar systems apply heavy compression (e.g. run-length, dictionary, delta encoding). Compression can speed I/O (reading less data). When inserting, systems may use a “wide” or “compact” format (ClickHouse switches based on part size 33). The choice of codec (ZSTD, LZ4, Snappy) and whether data is pre-sorted (dict-encoded) matters.

- **Min/Max (Zone) Indexes:** Many systems keep *min/max* values per data block or page to skip ranges quickly. Snowflake's micro-partitions store min/max of each column ²⁵, so queries with `col > X` skip partitions whose max < X. Similarly, ORC/Parquet files embed column min/max in row groups. ClickHouse's primary key ("marks") store the first row's key of each block, enabling range skipping ³⁴. PostgreSQL's BRIN index stores per-page-range summaries (e.g. min/max) for very large tables ³⁵: a BRIN on a timestamp can skip whole ranges of blocks if the filter falls outside the range. For example: "BRIN indexes handle very large tables with columns correlated with table order" ³⁵.
- **Skip Indexes and Bloom Filters:** ClickHouse supports *skip indexes* (a form of auxiliary index) on expressions, including Bloom filters. For example, adding a Bloom filter index on a column lets ClickHouse skip blocks that definitely lack certain values ³⁶. Cassandra uses Bloom filters per SSTable: on a read, Cassandra checks each SSTable's Bloom filter to see if the key "definitely does not exist" there, avoiding disk I/O ³⁷. Similarly, Hive/MR jobs sometimes build Bloom filters on join keys to filter partitions (the "Bloom join" optimization).
- **Local Indexes:** In partitioned setups, one can create local indexes per partition (e.g. PostgreSQL supports local indexes on each partition). Some columnar stores (BigQuery, Snowflake) lack user-visible indexes: instead they rely on partitioning/clustering and metadata pruning. BigQuery is "serverless" and auto-builds column stats for pruning. Snowflake automatically creates and maintains zone maps on micro-partitions ²⁵, so explicit indexes aren't needed.
- **Example (ClickHouse skip index):** A simple filter on `my_value` without a skip index scanned 800 MB and processed 100M rows ³⁸ ³⁹. After adding a set-based skip index and materializing it, the same query processed only 32K rows (~360 KB) ⁴⁰. The log showed dropping most data blocks ⁴¹. This illustrates how secondary indexes or skip filters can cut I/O by orders of magnitude.

Query Optimization Techniques

Distributed query engines employ many optimizations:

- **Predicate Pushdown:** Filtering conditions ("WHERE" clauses) are applied as early as possible, ideally down at the data read level. This reduces data volume upstream. For example, a Parquet reader will apply filters on column statistics and skip entire row-groups if possible. As Dremio notes, predicate pushdown "moves predicates directly to data sources, reducing the amount of data processed" ⁴². Combined with partition pruning, pushdown ensures only relevant data is read.
- **Partition Pruning (Partition Elimination):** If a table is partitioned, queries that filter on the partition column only scan matching partitions ⁴³. For example, BigQuery will only scan partitions whose date matches the filter ⁴³. Azure Synapse/Redshift and Spark/Hive also prune partitions if predicate matches partition key. This dramatically cuts I/O: e.g. scanning one day's data instead of a year's.
- **Clustering and Block Pruning:** Beyond partitions, columnar stores often sort or cluster data and keep block-level stats (min/max). BigQuery clustering and Snowflake clustering keys physically order data so that filters can prune blocks ²² ²⁵. For instance, BigQuery block pruning skips data blocks within a partition if the filter values don't fall in the block's range ⁴⁴.
- **Vectorized Execution:** Query engines process data in blocks (vectors) rather than row-at-a-time. ClickHouse explicitly uses a "vectorized execution model" (Processing blocks of thousands of rows with single CPU instruction streams) ⁴⁵. Spark's Tungsten engine and Trino/Presto similarly use vectorized processing on columnar formats. Vectorization reduces CPU overhead and boosts cache reuse.

- **Predicate Reordering and Early Bloom Filters:** During query planning, cheaper or more selective predicates can be applied first. Some systems build Bloom filters on the fly: for a join, after reading the small side, a Bloom filter is created and applied to streaming the large side (this is essentially a broadcast semi-filter before the join to drop non-matches early).
- **Cost-based vs Rule-based Planning:** Modern engines typically use cost-based optimization, considering table statistics to choose join orders and strategies. For example, Spark's Catalyst optimizer uses table statistics (from catalogs or file metadata) to estimate row counts ⁴⁶. It also has configuration (`spark.sql.autoBroadcastJoinThreshold`) controlling when to choose broadcast vs shuffle joins ²⁹. Older systems (Hive pre-Cost, some MySQL engines) used static rules. A cost-based planner can, for instance, choose to broadcast a table if statistics show it's below threshold.
- **Adaptive Query Execution (AQE):** Some systems (Spark 3.0+, Presto with adaptive execution) adjust the plan at runtime. E.g. Spark can split a shuffle into smaller tasks or switch to broadcast if data is smaller than estimated ³¹. This adaptivity helps on skewed data.

Pre-aggregation, Materialized Views, Approximate Algorithms

Aggregating trillions of rows on the fly can be expensive. Several strategies help:

- **Pre-aggregated Tables/Views:** Compute and store commonly-needed aggregates (rollups) ahead of time. For example, a table of daily summaries per country. These can be base tables or materialized views. BigQuery and Snowflake support *materialized views* that store query results. Query engines may automatically use (rewrite to) MVs when applicable. BigQuery notes that queries are “automatically rewritten to use materialized views whenever possible” for performance ⁴⁷. Snowflake's MVs are “automatically and transparently maintained” by background services ⁴⁸, providing up-to-date pre-computations. The trade-off is storage and maintenance cost vs faster query.
- **Incremental Aggregation:** Systems like Iceberg/Delta allow *merge-on-read* updates. As in the Ancestry example, hourly Spark jobs merged recent changes into the Iceberg table ⁴⁹. BigQuery MVs can incrementally add new rows if the base table only has appends ⁵⁰. Databases like Cassandra/Stargate often use “time-series bucketing” so incremental writes append to partitions. The idea is to avoid full re-compute by only updating affected partitions or keys.
- **Approximate Algorithms (Sketches):** For massive data, exact COUNT(DISTINCT) or quantiles can be slow. Approximate algorithms (with tunable error) give big speedups. BigQuery supports `APPROX_COUNT_DISTINCT` (HLL++) and explicit sketch functions (HLL++ INIT, MERGE, EXTRACT) ⁵¹. HyperLogLog (HLL) uses sublinear memory to estimate cardinalities. As the docs say, HLL “requires less memory than exact aggregation” and suits large data streams ⁵¹. Spark has similar `approxQuantile`, and Presto has `approx_distinct`. Using these, a COUNT(DISTINCT user_id) on billions of rows can be orders of magnitude faster, with minimal error.
- **Sketch Tables and Cubes:** Some systems (Druid, Cassandra with HyperLogLog sketches) maintain auxiliary data structures that speed up analytics. Techniques like *count-min sketch* or *quantile digest* are beyond scope but exist.

Query Routing and Execution Pruning

Distributed systems also optimize by routing queries:

- **Directed Query Routing:** If the query predicates fully specify a partition key, the coordinator can route the query to only the relevant nodes. E.g. in a sharded Postgres/Citus table, if `WHERE user_id = X` is given and `user_id` is the distribution key, the master can send the query only to the single shard holding that user. This eliminates querying all shards. Datastores like DynamoDB or MongoDB similarly route by hash key.
- **Partition Pruning (again):** As above, filtering on partition columns avoids full-table scans ⁴³. This often goes hand-in-hand with routing: e.g. Spark on Hive partitions prunes file lists; BigQuery prunes partitions.
- **Bloom Filter Joins:** Before a large join, engines can build a Bloom filter on the smaller side's join key, broadcast it, and filter the larger side's scan. This is effectively an in-memory index. The Ancestry Iceberg example did something analogous: in Spark's `MERGE INTO`, they first computed min/max ID to limit the join range ⁵², which enabled predicate pushdown and fewer shuffled rows. A Bloom filter would similarly drop many rows early.
- **Zone Maps (Column Statistics):** As noted, skip indexes and column stats (min/max) prune data. For example, Snowflake's pruning of micro-partitions is effectively zone maps ²⁵, and ClickHouse's primary key skipping is similar ⁵³. Query planners will incorporate these: any filter that matches an index or statistics can reduce data read by orders of magnitude.

Monitoring, Telemetry, and Benchmarking

Ensuring performance at scale requires monitoring and benchmarking:

- **Metrics:** Track query latency, throughput, CPU/Disk utilization, bytes scanned, and queue times. Cloud warehouses (BigQuery, Snowflake, Athena) provide query histories with scanned bytes and time. Clusters (Spark, Presto) offer metrics like executor CPU, memory spill, shuffle read/write. For example, BigQuery's `INFORMATION_SCHEMA` can show bytes processed per query. Snowflake's `QUERY_HISTORY` view reports execution time and bytes scanned per query.
- **Logging and Telemetry:** Capture query plans (`EXPLAIN`), job IDs, and logs. Use tools like AWS CloudWatch (Redshift, EMR), Databricks Ganglia metrics, or Prometheus exporters. Track slow queries, and correlate with cluster metrics to identify bottlenecks.
- **Synthetic Data:** Use benchmarks (TPC-DS, TPC-H, BigBench) to generate test datasets of desired scale. TPC-DS toolkit (dbgen) can produce up to petabytes for benchmarking. For time-series, tools like InfluxData's IBP or custom scripts can create realistic streams. Synthetic data should mimic skew and variety (e.g. zipf distribution).
- **Benchmark Workloads:** Run representative query sets (e.g. sample business queries or standard benchmarks) to measure performance. Track query planning time, execution time, and cost (for cloud billing). For example, using Databricks, one might run TPC-DS 10TB queries under different partitioning schemes to compare.
- **Cost Monitoring:** In cloud pay-as-you-go systems, monitor cost-per-byte. Techniques like partition pruning directly save money (BigQuery charges by scanned bytes). Use budget alerts on excessive scanning.

- **Alerts and Testing:** Set alerts for high latency or queue buildup. Automate periodic tests (nightly runs) to detect regressions.

Worked SQL Examples

Consider a table `events(date DATE, country STRING, user_id INT, action STRING)`. It has 50 billion rows partitioned by `date` (daily). Each daily partition is ~10 TB.

1. Bad Query (No Predicate):

```
SELECT country, COUNT(*)
FROM events
GROUP BY country;
```

Explanation: Scans all 50B rows across all dates. Very expensive – processes full data. No partitions are pruned.

1. Optimized (Date Filter):

```
SELECT country, COUNT(*)
FROM events
WHERE date = '2025-01-01'
GROUP BY country;
```

Improvement: Only the January 1, 2025 partition is scanned (10 TB instead of 500 TB). Partition pruning is applied. If further clustered by `country`, block pruning filters relevant country ranges ²². The explain plan would show only one partition scanned.

1. Bad Join (Different Keys):

```
SELECT o.order_id, c.country
FROM Orders o JOIN Customers c
ON o.order_date = c.signup_date;
```

Explanation: This joins on unrelated keys. The engine cannot broadcast either side (both large), so it must shuffle all orders by `order_date` and all customers by `signup_date` and match – a massive shuffle.

1. Optimized Join (Collocated Key): Suppose both tables are partitioned/sharded by `customer_id`, and we rewrite:

```
SELECT o.order_id, c.country
FROM Orders o JOIN Customers c
ON o.customer_id = c.customer_id;
```

If `Orders` and `Customers` share the same distribution/partition key, the join can be colocated. No full shuffle is needed: each shard/node can join its local subset of orders and customers. The explain plan would show a colocated or local join. This is common in sharded databases (e.g. Citus, Oracle Sharding). If one side is much smaller, an alternative is:

Broadcast Join Example (Spark syntax):

```
val df = spark.table("big_table")
val lookup = spark.table("small_table")
df.join(broadcast(lookup), "key").count()
```

Here, `small_table` is broadcasted to all executors ²⁸, and the join runs locally.

Technology Comparison (Summary)

System	Storage	Partitioning	Query Engine & Features	Scale & Use Case
PostgreSQL	Row-store (relational)	Range, List, Hash, Composite (native PG 10+) ¹⁷ ¹²	Single-master SQL; can do indexes (B-tree, BRIN) ³⁵ , No built-in sharding (use Citus for distribution)	OLTP and modest OLAP (up to ~10s of billions with sharding)
ClickHouse	Column-store (MergeTree)	Monthly/Yearly on date (PARTITION BY toYYYYMM) ¹⁸ ; ORDER BY as sort key ²⁰ ; supports skip indexes (minmax, bloom) ⁵⁴ ³⁶	MPP OLAP SQL (ANSI), vectorized engine ⁴⁵ , very fast scans/aggregation, customizable cluster shards ²⁰	Petabyte-scale OLAP (trillions of rows) for analytics, log/event data
BigQuery	Columnar (Dremel tree)	Time-column or integer range partitions ¹¹ ; clustering on up to 4 columns (user-defined) ²²	Serverless MPP SQL; auto-scaling; supports materialized views (incremental) ⁴⁷ , HLL++ sketches ⁵¹ , cost-based planning	Petabyte-scale analytics (cloud DW)
Snowflake	Columnar (micro-partitions)	Automatic micro-partitions; optional clustering keys (affects pruning) ²³	MPP SQL; auto-scalable; supports materialized views (auto-refresh) ⁴⁸ ; time-travel, data cloning	Petabyte-scale analytics (cloud DW)

System	Storage	Partitioning	Query Engine & Features	Scale & Use Case
Cassandra	Wide-row (LSM)	Partition key (hash), clustering columns for sort ¹⁵ ; secondary indexes possible, but wide row + partition key is main factor	CQL (NoSQL); eventually-consistent; Bloom filter per SSTable ³⁷ ; tunable consistency; best for high-write, time-series data	Very large writes/reads (hundreds of millions ops/sec across cluster); billions of rows possible
Spark/ Databricks	Mixed (file formats)	Depends on data source: e.g. Hive partitions on S3, or bucketing	MPP distributed compute; supports broadcast/shuffle joins ²⁸ , predicate pushdown to Parquet, cost-based AQE ³¹ , vectorized	Flexible scale (can run PB queries on files)
AWS Redshift / Azure Synapse	Columnar (cluster)	Distribution keys (hash, range); sort keys	MPP SQL; supports materialized views, result caching; automatic table re-sorting	Enterprise OLAP (hundreds of TB – PB)

(Scale & Use Case are illustrative; vendors often quote similar scales).

Recommendations and Conclusion

Query performance on billion+ row datasets hinges on minimizing unnecessary work. **Partition and shard data** so that common query predicates hit as little data as possible. For example, time-partition huge logs by day; hash-shard user data by user_id to spread load. **Co-locate join keys** when possible to avoid shuffles: pick a distribution key used in large joins. Use **columnar formats** with heavy compression for analytic workloads. Enable **predicate pushdown** (use Parquet/ORC) and **cluster sorted tables** to skip irrelevant data blocks ²² ⁴² . Choose **indexes and skip filters** wisely: ClickHouse skip indexes or PostgreSQL BRIN help prune by value, Cassandra's bloom filters avoid touching irrelevant SSTables ⁵⁴ ³⁷ .

Apply **vectorized execution engines** (most modern systems do) and use cost-based optimizers. Where aggregate queries dominate, consider **materialized views or pre-aggregations**. BigQuery or Snowflake MVs can speed up repeated queries on large tables ⁴⁷ ⁴⁸ . For extremely large DISTINCT or quantile queries, use **approximate algorithms** (HLL, sketches) to trade tiny error for huge speedups ⁵¹ .

Monitor everything: collect query and system metrics to spot bottlenecks. Benchmark with realistic data (e.g. TPC-DS, synthetic streams) to validate assumptions. The Ancestry case study shows that thoughtful partitioning (two-level by status/type), sorting (iceberg's truncate transform), and advanced features (Iceberg's storage-partitioned joins) can handle 100B rows and 7M updates with a 75% cost cut ² ⁵⁵ .

In conclusion, no single silver bullet exists; success comes from a **holistic approach**: designing the right architecture (data lake vs warehouse, batch vs stream), partitioning data for your workload, and exploiting engine features (pruning, indexing, joins) effectively. By following these best practices and continuously monitoring, teams can query multi-billion-row databases efficiently.

References: Authoritative vendor documentation and research were used throughout (BigQuery, Snowflake, ClickHouse docs [22](#) [23](#) [56](#) ; Apache Cassandra/PG docs [15](#) [35](#) ; AWS/Databricks case studies [1](#) [55](#) ; Spark SQL guide [57](#) [28](#)). Further details and examples are drawn from these sources.

[1](#) [2](#) [10](#) [26](#) [30](#) [49](#) [52](#) [55](#) How Ancestry optimizes a 100-billion-row Iceberg table | AWS Big Data Blog
<https://aws.amazon.com/blogs/big-data/how-ancestry-optimizes-a-100-billion-row-iceberg-table/>

[3](#) [6](#) [7](#) [8](#) OLTP vs OLAP - Difference Between Data Processing Systems - AWS
<https://aws.amazon.com/compare/the-difference-between-olap-and-oltp/>

[4](#) [5](#) [9](#) Kappa vs Lambda Architecture: A detailed comparison (2026)
<https://www.flexera.com/blog/finops/kappa-vs-lambda-architecture/>

[11](#) [21](#) [27](#) Introduction to partitioned tables | BigQuery | Google Cloud Documentation
<https://docs.cloud.google.com/bigquery/docs/partitioned-tables>

[12](#) [13](#) [17](#) PostgreSQL Table Partitioning Using Traditional Methods
<https://www.percona.com/blog/postgresql-partitioning-using-traditional-methods/>

[14](#) [18](#) [19](#) [20](#) [32](#) [33](#) [34](#) [53](#) [56](#) MergeTree table engine | ClickHouse Docs
<https://clickhouse.com/docs/engines/table-engines/mergetree-family/mergetree>

[15](#) [16](#) Define and optimize data partitions in Apache Cassandra | Opensource.com
<https://opensource.com/article/20/5/apache-cassandra>

[22](#) [44](#) Introduction to clustered tables | BigQuery | Google Cloud Documentation
<https://docs.cloud.google.com/bigquery/docs/clustered-tables>

[23](#) [24](#) [25](#) Micro-partitions & Data Clustering | Snowflake Documentation
<https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions>

[28](#) Broadcast hash join - Microsoft Q&A
<https://learn.microsoft.com/en-us/answers/questions/1508214/broadcast-hash-join>

[29](#) [31](#) [46](#) [57](#) Performance Tuning - Spark 4.1.1 Documentation
<https://spark.apache.org/docs/latest/sql-performance-tuning.html>

[35](#) PostgreSQL: Documentation: 18: 65.5. BRIN Indexes
<https://www.postgresql.org/docs/current/brin.html>

[36](#) [38](#) [39](#) [40](#) [41](#) [54](#) Understanding ClickHouse data skipping indexes | ClickHouse Docs
<https://clickhouse.com/docs/optimize/skipping-indexes>

[37](#) Bloom Filters | Apache Cassandra Documentation
https://cassandra.apache.org/doc/latest/cassandra/managing/operating/bloom_filters.html

[42](#) What is Predicate Pushdown? | Dremio
<https://www.dremio.com/wiki/predicate-pushdown/>

43 Query partitioned tables | BigQuery | Google Cloud Documentation

<https://docs.cloud.google.com/bigquery/docs/querying-partitioned-tables>

45 Architecture overview | ClickHouse Docs

https://clickhouse.com/docs/academic_overview

47 50 Use materialized views | BigQuery | Google Cloud Documentation

<https://docs.cloud.google.com/bigquery/docs/materialized-views-use>

48 Working with Materialized Views | Snowflake Documentation

<https://docs.snowflake.com/en/user-guide/views-materialized>

51 HyperLogLog++ functions | BigQuery | Google Cloud Documentation

https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/hll_functions